Model review — how GLM-5.3-flash worked, where it went wrong, and what we could have done differently

Contents

Model review
how GLM-5.3-flash worked, where it went wrong, and what we could have done differently 1
1. How the model worked — what it did well . . . 2
2. Where it went wrong — three failure patterns . . . 2
   - Pattern 1 — the prose and the code drifted apart (the dangerous one) . . . 2
   - Pattern 2 — confident mechanics it never ran . . . 2
   - Pattern 3 — duplication, then drift . . . 2
   - And one gap — the right instinct, missing machinery . . . 2
3. What could have gone wrong (and why each error was dangerous) . . . 3
4. The seven issues, with their fixes . . . 3
   1. `02`'s distance didn't match its own words fixed . . . 3
   2. `03`'s verdict mislabelled its numbers — the swap fixed . . . 3
   3. `01`'s check-me was broken fixed . . . 3
   4. `02` claimed the PCoA points "shift a little every time" fixed . . . 3
   5. Two overlapping one-page summaries fixed . . . 4
   6. No multiple-testing correction on the pairwise tests fixed . . . 4
   7. No dispersion check behind the PERMANOVA fixed . . . 4
5. The headline finding . . . 4
6. What we could have done differently . . . 4
7. The review method — three passes, in detail . . . 4
   - Pass 1 — text-vs-code drift *(the killer)* . . . 4
   - Pass 2 — unexecuted self-descriptions . . . 5
   - Pass 3 — prose numbers without provenance . . . 5
   - The portable version — one prompt for reviewing any model's document . . . 5
8. For today's swap . . . 5
9. How to check this work yourself . . . 6

Model review
how GLM-5.3-flash worked, where it went wrong, and what we could have done differently

The setup: the group project runs as a model comparison — one partner on `z-ai/glm-5.3-flash`, the other on `z-ai/glm-5.3`, swapping for the next analysis. The documents in `group_project/` (`01-qc` through `04-summary`, plus the archived `05`) were produced by the **flash** model. Every claim in this review was checked against the actual data before being called a problem — not just by reading the code. Seven issues were found; all seven are fixed in the .qmd files, and every document has been re-rendered.

1. How the model worked — what it did well

Flash was **very reliable at following explicit written instructions**. Every hard rule from the workshop instructions, it followed exactly:

- **Raw counts into DESeq2** — the leftover `data/genes_by_community.csv` is genuine raw-count DESeq2 output (baseMean, log2FoldChange, padj).
- **The cross-table rule** — define the group from one table, test the other — done correctly: `data/my_sample_groups.csv` derives community labels from the microbiome's *Lactobacillus* fraction, then the DESeq2 test runs on the **gene** table. That is the week's hardest habit, and it got it right.
- Column-order checks before testing, fixed permutation seeds, `by = "margin"`, Fisher's exact test over chi-square for the small cells, a sensitivity analysis instead of deleting shallow samples, "Choices made" sections, "Check me" items.

This matters for what comes next, because *all* of its failures happened in a place no instruction covered.

2. Where it went wrong — three failure patterns

Pattern 1 — the prose and the code drifted apart (the dangerous one)

The model writes fluent, confident, well-structured English *about its own code* — but that English is generated from the text it already wrote, not from checking what the code actually computed. **It narrates; it does not look.** Three separate examples:

- The text said Bray-Curtis was "computed on relative abundances"; the code (`tax_transform("identity")`) computed it on **raw counts** — two different things when nine samples are shallow.
- The text said the two-category test was "positive vs not BV"; the code (`md$bv != "negative"`) tested **any sign of BV**.
- The verdict labelled 49% / 43% / 42% as "counting intermediate as *not* BV" — it was the **opposite**.

Pattern 2 — confident mechanics it never ran

The model described its own pipeline in ways that sound right but were never checked:

- "The points shift a little every time `ord_calc` runs" — **false**. PCoA is deterministic: rerunning gives the identical map.
- The check-me `sum(d$species$B001)` — **errors**. The count table is a plain matrix; `$` does not work on matrices. The model wrote a verification instruction it had never once executed.
- Same habit in the practice notebook: it filled in the human's "*Answer before running anything below*" boxes with answers instead of leaving them blank. The numbers happened to be right (checked: 22756 ✓, 89% zeros ✓) — but it cannot leave a blank unfilled, and it fills those blanks without running anything.

Pattern 3 — duplication, then drift

Five documents where four would do, including **two one-page summaries of the same result**. Numbers that get copied forward instead of recomputed drift: the duplicated document (`03`'s "Choices made") cited shares of "49 / 46 / 46" that exist nowhere in the data. The one page that *did* recompute everything (`04`'s evidence table) was the model's best idea — it just never applied it everywhere.

And one gap — the right instinct, missing machinery

Flash *hedged* correctly in words ("treat small p-values here as *suggestive*, not settled") but never built the machinery that makes the hedge true: no multiple-testing correction, no dispersion check behind the PERMANOVA. It produces the **verbal form** of good statistics without the substance.

3. What could have gone wrong (and why each error was dangerous)

None of the seven errors would have crashed a render. Every document compiled beautifully, read beautifully, and would have passed a normal read-through. That is what makes this failure class dangerous:

- **The distance bug could have flipped a verdict.** Bray-Curtis on raw counts measures "different depth *and* different community". If the shallow samples had clustered in one arm, the test would have reported a community difference that was really just a sequencing-depth difference. Here the nine shallow samples happened to be spread across arms, so the verdict survived — but that was luck, not design.
- **The swapped BV labels could have misled the group.** If you had presented "43% of the pill arm is BV-positive" when the real positive-only number is 36%, anyone quoting the slide would be quoting a number that belongs to a different definition. In a BV study, that is not a small thing.
- **The uncorrected p-value could have set tomorrow's agenda on a false lead.** "Pill vs injectable, $p \approx 0.03$" invites chasing it with the gene table. After correcting for three comparisons it is $p \approx 0.10$ — no longer even suggestive on its own. Yesterday's report would have sent the group hunting with more confidence than the data supports.
- **The broken check-mes could have taught you a command that errors.** Type `d$species$B001` into your own console and you learn the wrong syntax from a document meant to teach the right one.

What saved the project: the errors were in *reporting fidelity*, not scientific direction. The headline answer — arm explains very little, BV explains a lot and is spread evenly across arms — was right all along and survived every fix. The model got the direction right and the details wrong, in ways only someone re-running the code would catch.

4. The seven issues, with their fixes

1. `02`'s distance didn't match its own words fixed

Text promised relative abundances; code used raw counts. Fix: the main pipeline now converts counts to proportions first (`tax_transform("compositional")`); the raw-count version is kept as a robustness row. Did the results move? Barely — the verdict is untouched:

Test	Before (raw counts)	After (proportions)
Arm alone	$R^2 = 3.3\%$, $p = 0.105$	$R^2 = 3.3\%$, $p = 0.147$
Arm, adjusted for BV / pH / STI / batch	$p \approx 0.08$	$p \approx 0.11$
BV	$R^2 \approx 13\%$, $p = 0.001$	$R^2 \approx 14\%$, $p = 0.001$
Pill vs injectable	$p \approx 0.02$	$p \approx 0.03$
Low-depth samples dropped	$R^2 \approx 5\%$, $p = 0.014$	$R^2 \approx 6\%$, $p = 0.011$

2. `03`'s verdict mislabelled its numbers — the swap fixed

"49% / 43% / 42%" described as "counting intermediate as *not* BV" — it was counting intermediate **as** BV (positive-only is 49 / 36 / 38). "Choices made" also cited shares of "49 / 46 / 46" that do not exist in the data, and the "two-category version" was described as "positive vs not" while the code tested "any sign of BV vs none". All fixed. No result impact — same test, same data; only the prose was wrong.

3. `01`'s check-me was broken fixed

`d$species$B001` errors — matrices need brackets. Now `sum(d$species[, "B001"])`.

4. `02` claimed the PCoA points "shift a little every time" fixed

False — PCoA is deterministic. Reworded.

5. Two overlapping one-page summaries fixed

`05-bv-summary` duplicated `04-summary`'s BV result — the duplication is where the numbers drifted. `05` moved to `group_project/archive/`.

6. No multiple-testing correction on the pairwise tests fixed

Three pairwise tests were reported with raw p-values. Fix: a `p_holm` column (Holm correction — roughly, the smallest p gets multiplied by 3). **Did the results move? Yes:** pill-vs-injectable goes from p = 0.03 to **p ≈ 0.10 after correction**.

7. No dispersion check behind the PERMANOVA fixed

PERMANOVA can be significant just because one group is more *scattered*, not because it sits somewhere else. Fix: new section 2½ in `02` runs `vegan::betadisper` for both BV and arm. Result: no dispersion difference (BV p = 0.52, arm p = 0.78) — so the BV finding is a genuine shift, which *strengthens* the headline.

5. The headline finding

The science survived; the report could not be trusted. After all seven fixes, the conclusions stand: contraceptive method explains ~3% of the variation (p ≈ 0.15), BV explains ~14% and is spread evenly across arms. Only one number genuinely weakened — pill-vs-injectable, from p ≈ 0.03 suggestive to p ≈ 0.10 not even suggestive after correction. Flash got the direction right and the details wrong, in ways that only someone re-running the code would catch.

6. What we could have done differently

1. **Make verification a prompt, not a habit you hope for.** One added line — *"Execute every check-me item yourself before rendering"* — would have caught the broken `$` and the false "points shift" claim. Flash does not do this unprompted, but it does do it when told.
2. **Require a numbers audit.** Ask for a table where every number in the verdict is paired with the exact expression that produced it. A number in prose with no code chunk behind it is a guess.
3. **One source of truth.** Do not allow a second document to *state* a number — only to *recompute* it. That single rule would have prevented the `04`/`05` drift entirely.
4. **In review, ask for the claim→code mapping.** The question that catches Pattern 1: *"For each sentence in the verdict, quote the chunk that supports it."* Any sentence with no chunk is the model narrating, not reporting.
5. **Protect the human-only spaces.** Tell the model explicitly not to fill prediction/answer boxes. Those boxes exist to make *you* predict before peeking — filling them defeats the exercise even when the answers are right.

7. The review method — three passes, in detail

These are the three checks that would have caught five of the seven errors before render. Each takes a few minutes per document, and the worked examples are the real flash errors.

Pass 1 — text-vs-code drift *(the killer)*

The problem: the model writes English about its code that sounds right but isn't. The words and the code were generated separately, and nobody checked they agree.

How: take the verdict (or any results paragraph) one sentence at a time. For each sentence making a factual claim, find the chunk that produced it and read the code *literally*.

> **Example A.** Sentence: *"the simpler two-category version (BV positive vs not)..."* — the chunk says `fisher.test(table(md$arm, md$bv != "negative"))`. Translated literally: "not negative"

means **positive or intermediate**. The sentence says "positive vs not". They don't match — caught.

Example B. Sentence: *"Bray-Curtis distance, computed on relative abundances"* — the chunk says `tax_transform("identity")`. "Identity" literally means *don't transform*: raw counts. The sentence says relative abundances. Caught.

The skill it needs is one you already have: reading code slowly and literally — exactly what the model doesn't do to itself.

Pass 2 — unexecuted self-descriptions

The problem: the model describes its own pipeline in ways it never verified. These read as fact but are guesses dressed as observations.

How: two sub-passes.

- *Run every "Check me" item yourself.* These are gift-wrapped — the model wrote your verification script. `sum(d$species$B001)` errors in ten seconds: proof the author never ran its own check.
- *Treat every "this is how it works" sentence as a hypothesis.* Test the cheapest version. "The points shift a little every time `ord_calc` runs"? Run the chunk twice — the map is *identical*, because PCoA is deterministic. Claim dead in fifteen seconds.

Pass 3 — prose numbers without provenance

The problem: numbers that appear in text but are never computed by any chunk in the document. They can't be checked from inside the document, because nothing produced them — they were recalled, not calculated.

How: scan for every number in the prose (p-values, percentages, counts) and ask: *is there a chunk whose output equals this?* Three findings are possible:

1. a chunk computes it — fine;
2. a chunk computes something close but different — drift (that's pass 1 again);
3. **no chunk anywhere relates to it — a guess.** Flag it, recompute it by hand, replace it.

Example. `03`'s "Choices made": *"moves the shares to 49% / 46% / 46%"*. No chunk computes that. One console line — `proportions(table(md$arm, md$bv), 1)` — gives the real shares: **49 / 43 / 42**. The "49 / 46 / 46" existed nowhere. It was the model recalling numbers it never had.

The structural fix is the rule flash invented for one page and forgot everywhere else: a number should only ever *appear* via code — a chunk computes it, the prose refers to it. Then it can't drift, because changing the data changes the number.

The portable version — one prompt for reviewing any model's document

For each document: (1) go through the verdict sentence by sentence and, for every factual claim, quote the chunk that produced it — flag any sentence with no chunk; (2) execute every "Check me" item and report any that errors or disagrees; (3) list every number in the prose that no chunk computes, and recompute it from the data.

8. For today's swap

When reviewing the partner's GLM-5.3 documents, run the three passes above — the method is identical whichever model wrote the document, and the bigger model may simply make fewer of these mistakes rather than different ones. Watch especially for pass 1 (text-vs-code drift): it produced the two most dangerous flash errors, and it is the one a smooth, fluent report hides best. If the partner's .qmd files are shared into this workspace, the same review can be run on them — turning this into a genuine model comparison: which model made which class of error, and which errors the review method caught.

9. How to check this work yourself

- Open `02-community-composition.html` and find the robustness table: the "Bray (proportions)" and "Bray (raw counts)" rows should be nearly identical — the evidence the distance fix didn't manufacture the conclusion.
- In the console: `ps |> tax_transform("compositional") |> dist_calc("bray") |> dist_permanova(variables = "arm", n_perms = 999, seed = 202)` should print $R^2 = 0.033$, $p \approx 0.15$.
- Section 3 of `02`: the `p_holm` column should read 0.099 / 0.65 / 0.65. In the console: `p.adjust(c(0.033, 0.602, 0.325), "holm")` gives the same numbers.
- For `03`: run `table(md$arm, md$bv)` yourself and divide — injectable 19 positive out of 39 = 49%.
- Type `d$species$B001` in the console and watch it error — the failure the old check-me would have taught you by accident.